

COMP1531

Software Engineering Fundamentals

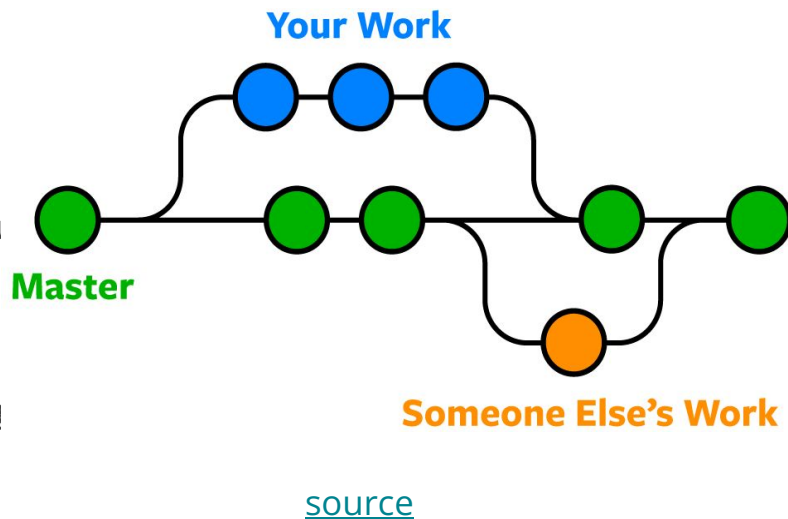
Week 1
Git - Team Usage

In this Lecture

- Git is primarily useful when working with others, and working with others effectively is important.

What Makes That Possible?

- Branching
 - Work on features independently
 - No breaking the main project
 - Safe experimentation
- Merging
 - Combine changes from different contributors
 - Keep progress unified
 - Resolve conflicts collaboratively
- Merge Requests
 - Structured code review before changes
 - Improve quality/ Share Knowledge
 - Catch bugs early



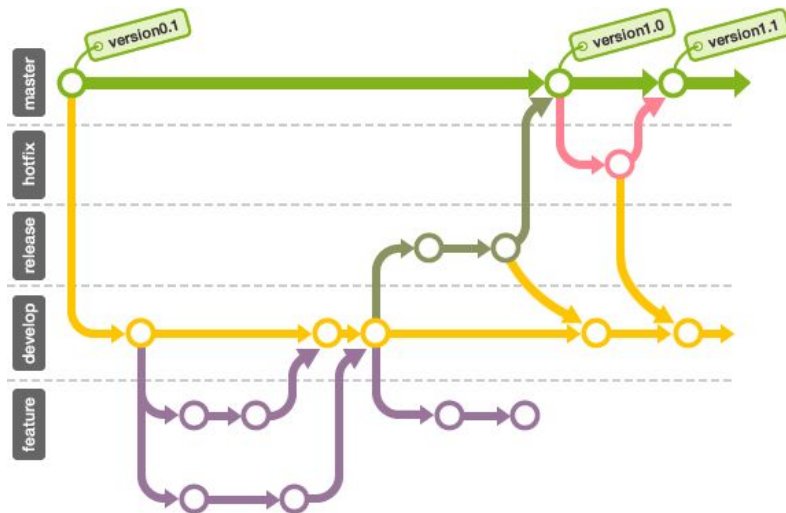
-- --

Most of today's explanations will be covered via a live demo. If you want to follow a written guide, then please checkout [Atlassian's git guide](#)

The Git Tree Model

- Git can be understood as a tree-like structure.
- Git is a collection of commits.
- Each commit has one parent. Each commit can have multiple children (i.e. branches)

- A branch essentially is just a pointer to a particular commit.
- To try and bring two separate branches together onto the same commit is a process of "merging"



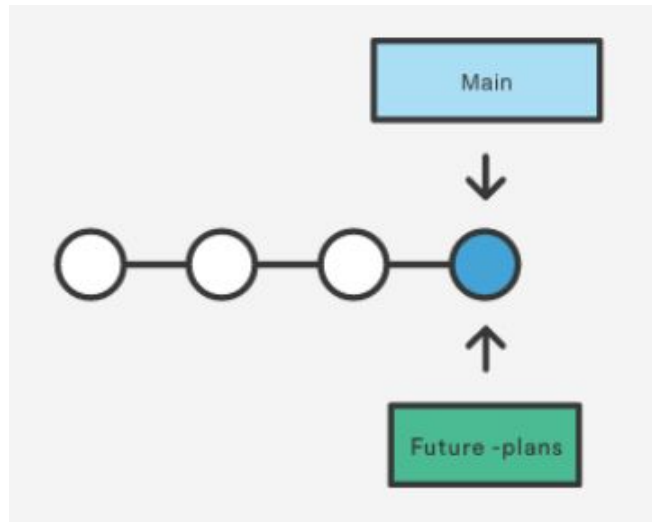
[source](#)

Branches

- Your "master" branch is just a pointer to a particular commit on the tree (usually the latest).
- You can create your own branch if you want to continue on a separate thread of working, unrelated to the master branch.

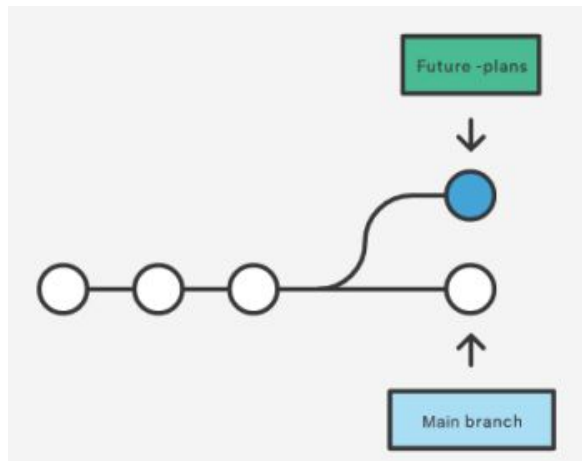
```
1 git checkout -b new_branch_name
```

Source: Atlassian Git Guide



Branches

- This then allows you to continue making commits on a separate "branch".
- There is no limit for the number of branches you can have in a repository.



Source: Atlassian Git Guide

Branches

- Your local repository can only "check out" (work with) a single branch at a time. You can swap between branches using the checkout command.
- It's generally good practice to ensure you have no staged or unstaged changes on your branch before swapping to another.

```
1 git checkout branch_to_swap_to
```

Merging

The process of "incorporating work on another branch into mine" is known as merging.

The Two Most Common Merge Scenarios:

1. While you're developing, you regularly merge the latest updates from main so you stay up to date with bug fixes , small improvements Other team members' changes

👉 Keeps your branch current and reduces big conflicts later.

2. When your work is complete and reviewed, you merge your branch back into main.

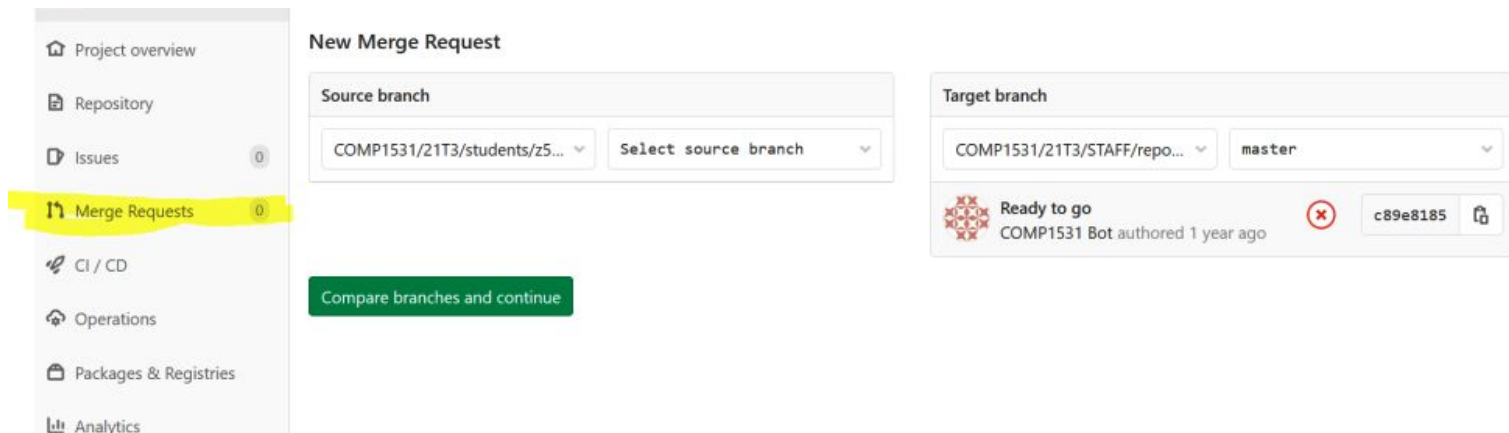
```
1 git merge master
```

👉 This integrates your finished feature into the project.

Merge Requests

You Don't Merge to **main** Directly

- In most professional environments, you **cannot merge your branch into **main** (or **master**) via the command line.**
- Instead, we allow our git site to do it via a Merge Request (a web-based GUI that helps manage merges into master)



We Use a Merge Request

— — —

A **Merge Request (MR)** is a web-based workflow (e.g., in GitLab) that:

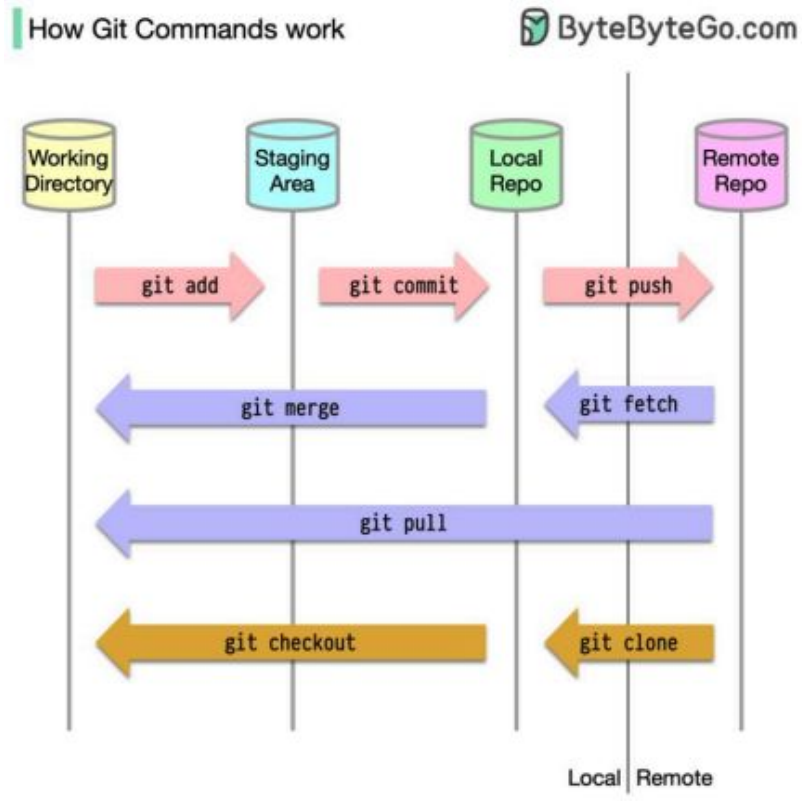
- Shows the changes you made
- Allows others to review your code
- Enables discussion and feedback
- Runs automated tests (CI/CD)
- Controls approval before merging

Why This Matters?

- Protect the main branch
- Improve code quality
- Encourage collaboration
- Prevent accidental breakage

How Git Commands Work

— — —



Feedback

— — —



Or go to the [form here](#)